

Health checks & logging

Answer "are you OK?" honestly, and write down what went wrong when you're not.

The big idea

In production nobody watches the app. Other programs visit `/health/` every few seconds and ask "are you OK?". A **load balancer** stops sending visitors to a copy that says no.

Like a nurse checking a pulse, asking isn't enough: you have to **measure**. And when it fails, write the details into the **log**, not the answer.

What your code receives and returns

The tests build a fake `GET /health/` request and call your view directly.

```
response = views.health(factory.get("/health/"))

# database works (test 1):    200
{"status": "ok", "checks": {"database": "ok"}}

# database down (tests 2-5): 503, and an ERROR log line with the traceback
{"status": "error", "checks": {"database": "error"}}

# POST /health/ (test 6):    405
```

New words

health check

An address other programs call to ask "is this server working?".

load balancer

Shares incoming visitors between several copies of your server.

dependency

Something your app needs to work, like the database.

logger

The object you write log lines with: `logger.error("...")`.

log level

How serious a line is: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`.

traceback

The "Traceback (most recent call last)" report of where an error happened.

Why 503, and not 200 or 500?

- **2xx** success, **4xx** the client's fault, **5xx** the server's problem
- **200** would be a lie: visitors keep going to a broken server
- **500** usually means "my code crashed": a bug
- **503** means "I'm alive, but a dependency is down. Try elsewhere."
- Load balancers read the **number**, not the text

Tools: a cursor, and the DatabaseError family

`SELECT` a tiny sum to prove the database answers; `DatabaseError` catches all database errors.

```
from django.db import DatabaseError, OperationalError, connection

with connection.cursor() as cursor:
    cursor.execute("SELECT 2 + 3")
    print(cursor.fetchone())

print(issubclass(OperationalError, DatabaseError))    # part of the family
```

OUTPUT

(5,)

True

Tool: loggers and levels

Without extra setup, only **WARNING** and more serious lines are shown.

```
import logging

log = logging.getLogger("shop")
log.info("A customer opened the shop")    # not shown: below WARNING
log.warning("Only 2 cups of tea left")
log.error("The till is broken")
```

OUTPUT

```
Only 2 cups of tea left
The till is broken
```

Tool: `logger.exception` = message + traceback

```
try:
    price = int("ten")
except ValueError:
    log.exception("Could not read the price")
print("the program keeps going")
```

OUTPUT

```
Could not read the price
Traceback (most recent call last):
  File "<selection>", line 5, in <module>
ValueError: invalid literal for int() with base 10: 'ten'
the program keeps going
```


logger.error vs logger.exception

logger.error(...)

- Writes an **ERROR** line
- Only your message, **no traceback**
- Works anywhere
- Test 4 fails: nothing shows where it broke

logger.exception(...)

- Writes an **ERROR** line
- Adds the **traceback** automatically
- Use it only inside **except**
- What tests 3 and 4 look for

Tools: JsonResponse and @require_POST

```
from django.http import JsonResponse
from django.test import RequestFactory
from django.views.decorators.http import require_POST

print(JsonResponse({"tea": "hot"}, status=202).status_code)    # status= sets the number

@require_POST          # a decorator: other methods get 405
def ping(request):
    return JsonResponse({"reply": "pong"})

print(ping(RequestFactory().get("/ping/")).status_code)
```

OUTPUT

202

405

Pattern: try, log, answer safely (steps 2-4)

```
def stock_check(count_text):  
    try:  
        count = int(count_text)  
    except ValueError:  
        log.exception("Stock check failed")  
        return {"status": "error"}  
    return {"status": "ok", "count": count}  
  
print(stock_check("lots"))    # stock_check("12") gives {'status': 'ok', 'count': 12}
```

OUTPUT

```
Stock check failed  
Traceback (most recent call last): ...  
ValueError: invalid literal for int() with base 10: 'lots'  
{'status': 'error'}
```

Your task, step by step

- 1 Put `@require_GET` on the line directly above `def health(request):` → **test 6**
- 2 In `try:` open with `connection.cursor()` as `cursor:` and run `SELECT 1`
- 3 `except DatabaseError:` call `logger.exception(...)` with your own message → **test 4**
- 4 Then return the error dict, written by hand, with `status=503` → **tests 2, 3, 5**
- 5 After the `try/except`, return the ok dict (status 200) → **test 1**

Common mistakes

- Answering ok without a query: test 2 notices
- `connection.cursor()` outside the `try`: the error escapes
- Importing `connection` again inside the function: the fake is skipped
- `logger.error` instead of `logger.exception`: no traceback
- `str(error)` in the response: it leaks `password=hunter2`

Where you'll use this

You run three copies of your app behind a load balancer. One loses its database connection at 3 a.m. Its `/health/` answers 503, visitors move to the other two, and in the morning the log's traceback tells you why.

- Kubernetes, Render, Railway and AWS all let you set a health check URL
- They act on the **status code**, so it has to be right
- Logs are how you fix problems you can't reproduce on your laptop

Check yourself

1. Why is a health check that always says ok useless?
2. Why answer 503 rather than 200 or 500 when the database is down?
3. What does `logger.exception` add that `logger.error` doesn't?
4. Why must the error message stay out of the response?

Keep learning

[What Are Logs in Programming?](#)

freeCodeCamp · article · beginner. Start here: what logs are for and the five levels (DEBUG to CRITICAL), with Python logging examples.

[Logging HOWTO](#)

Python docs · tutorial · beginner. The official beginner tutorial for logging.getLogger(__name__) and logger.exception, used when the database check fails.

[503 Service Unavailable](#)

MDN Web Docs · docs · beginner. What the 503 status means and why a health endpoint returns it when the database is down.

[Performing raw SQL queries: executing custom SQL directly](#)

Django docs · docs · intermediate. Shows connection.cursor() and cursor.execute, which your health check uses to run a tiny 'SELECT 1' test query.

[Health Endpoint Monitoring pattern](#)

Microsoft Learn · article · intermediate. The bigger picture: how monitoring tools call /health, what to check (like the database) and how to keep it safe.